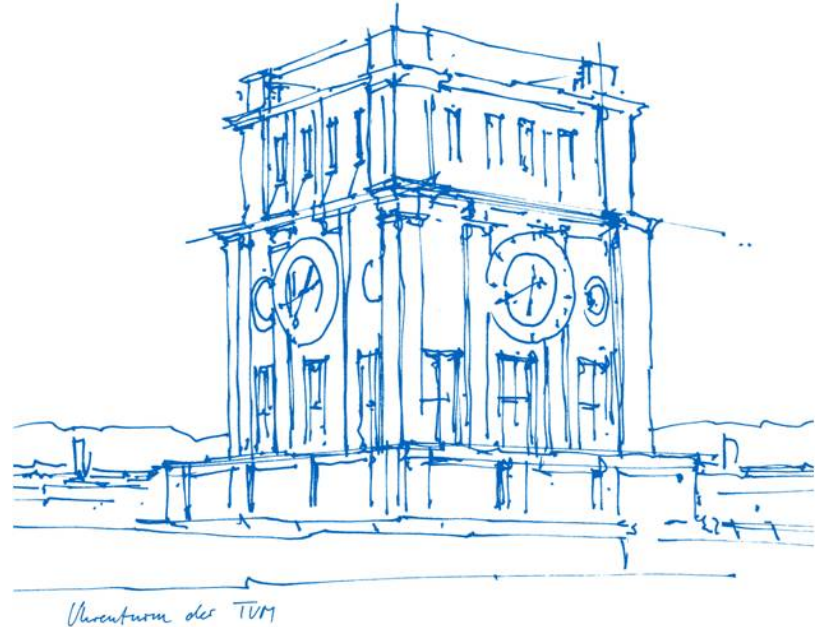


On the Formal Verification of Polynomial Commitments: two KZG constructions and the Algebraic Group Model

Tobias Rothmann

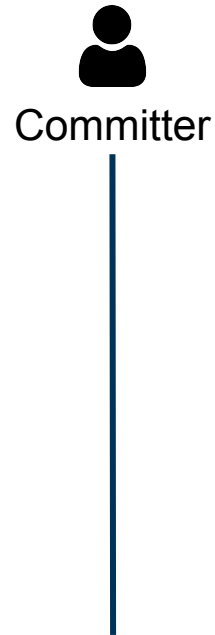
Technische Universität München

Rome, 14th September, 2026

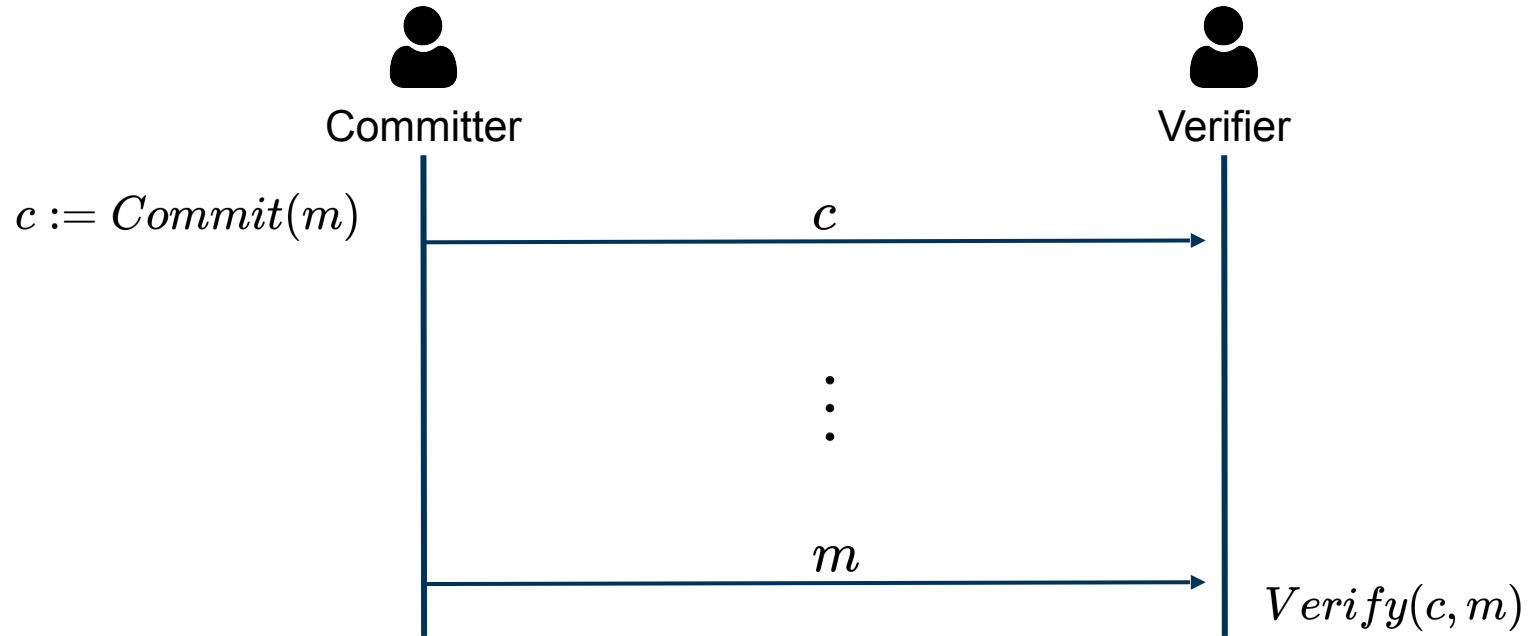


Recall: Polynomial Commitments

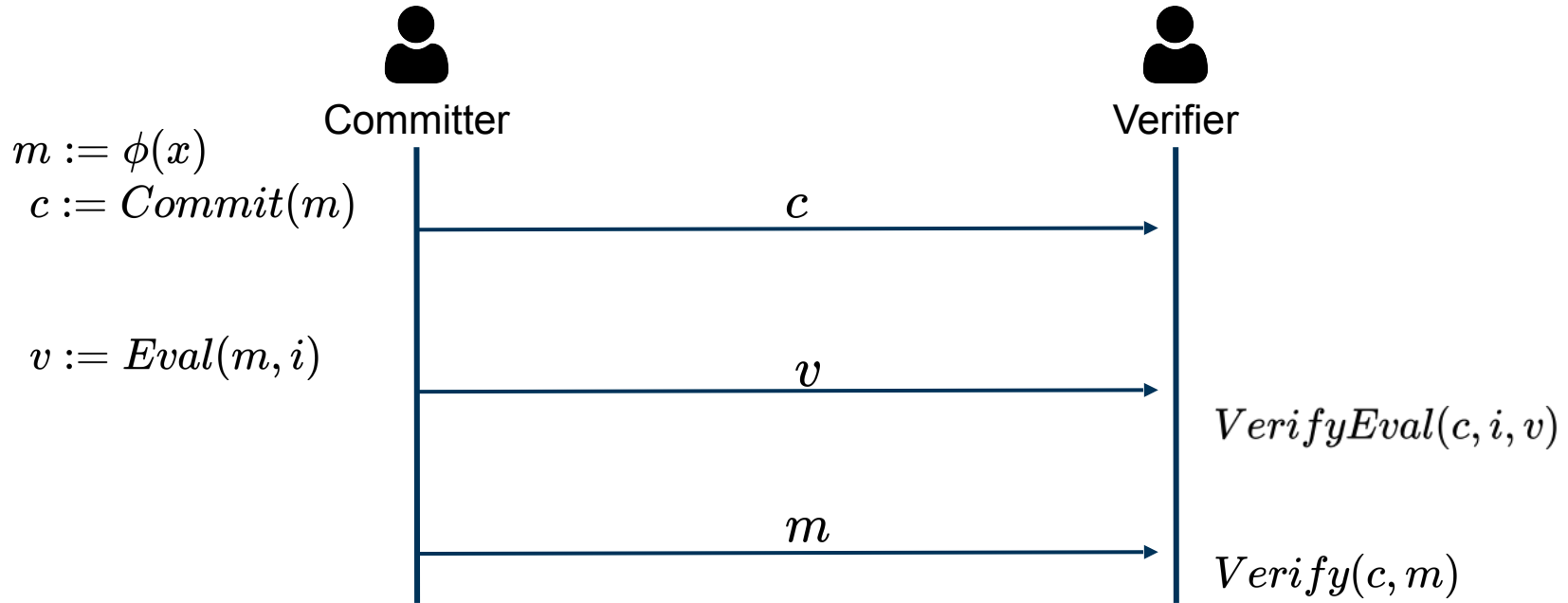
Recall: Polynomial Commitments



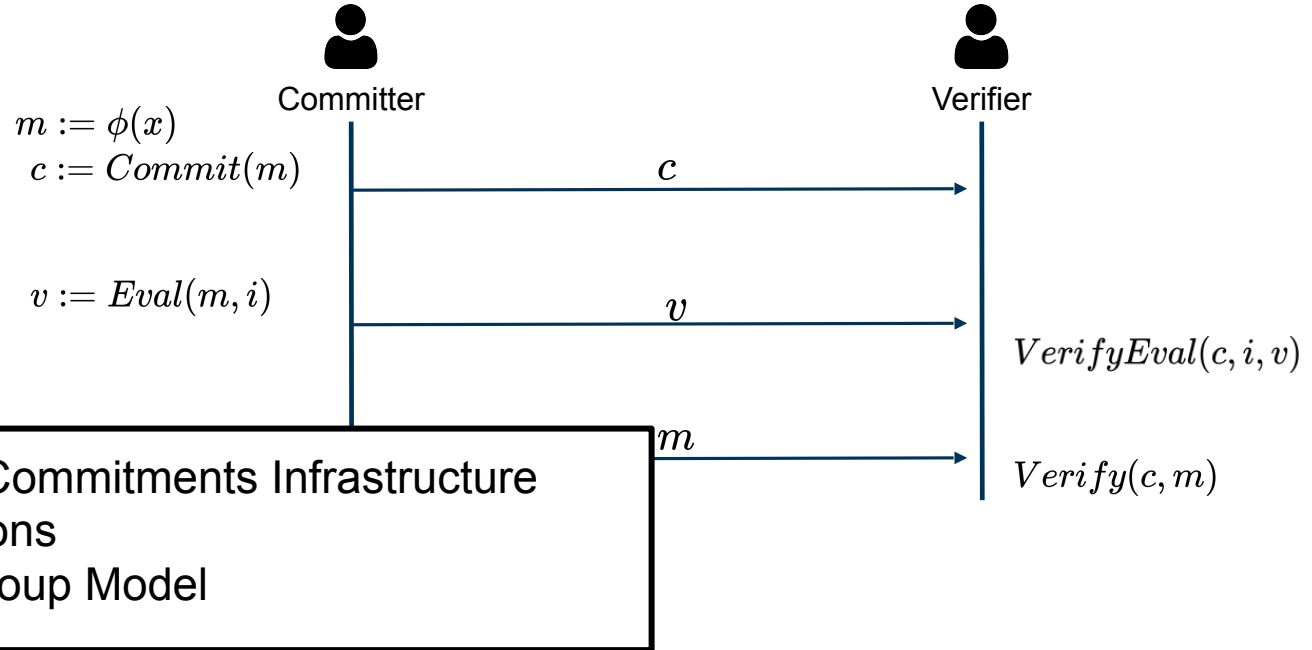
Recall: Polynomial Commitments



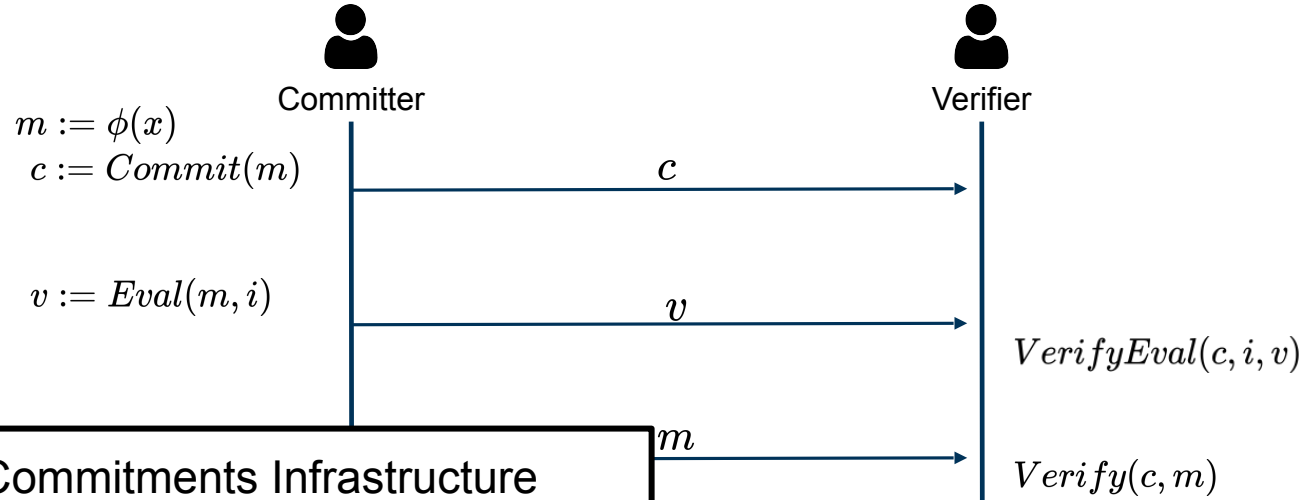
Recall: Polynomial Commitments



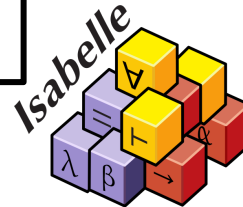
This Work



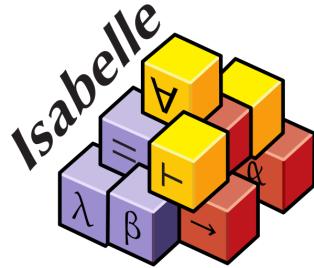
This Work



1. Polynomial Commitments Infrastructure
2. 2 KZG versions
3. Algebraic Group Model



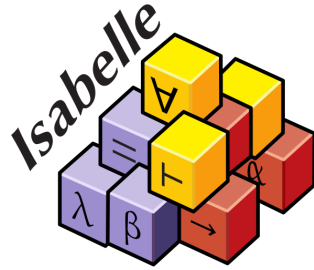
Isabelle proof assistant



Isabelle proof assistant



Isabelle proof assistant

**Isabelle:**

- Checks general mathematical proofs
- Higher Order Logic (HOL)
- small logical kernel



Formalizing Polynomial Commitments

```

locale abstract_polynomial_commitment_scheme =
  fixes key_gen :: "('ck × 'vk) spmf" — <outputs the keys received by the two parties>
  and commit :: "'ck ⇒ 'r::zero poly ⇒ ('commit × 'trapdoor) spmf"
    — <outputs the commitment as well as the secret, which might be used to derive witnesses,
      and the opening values sent by the committer in the reveal phase>
  and verify_poly :: "'vk ⇒ 'r poly ⇒ 'commit ⇒ 'trapdoor ⇒ bool"
    — <checks whether the polynomial corresponds to the commitment>
  and eval :: "'ck ⇒ 'trapdoor ⇒ 'r poly ⇒ 'argument ⇒ ('evaluation × 'witness)"
    — <outputs a point and a witness>
  and verify_eval :: "'vk ⇒ 'commit ⇒ 'argument ⇒ ('evaluation × 'witness) ⇒ bool"
    — <checks whether the point is on the polynomial corresponding to the commitment>
begin

```

Formalizing Polynomial Commitments

```

locale abstract_polynomial_commitment_scheme =
  fixes key_gen :: "('ck × 'vk) spmf" — <outputs the keys received by the two parties>
  and commit :: "'ck ⇒ 'r::zero poly ⇒ ('commit × 'trapdoor) spmf"
    — <outputs the commitment as well as the secret, which might be used to derive witnesses,
    and the opening values sent by the committer in the reveal phase>
  and verify_poly :: "'vk ⇒ 'r poly ⇒ 'commit ⇒ 'trapdoor ⇒ bool"
    — <checks whether the polynomial corresponds to the commitment>
  and eval :: "'ck ⇒ 'trapdoor ⇒ 'r poly ⇒ 'argument ⇒ ('evaluation × 'witness)"
    — <outputs a point and a witness>
  and verify_eval :: "'vk ⇒ 'commit ⇒ 'argument ⇒ ('evaluation × 'witness) ⇒ bool"
    — <checks whether the point is on the polynomial corresponding to the commitment>
begin

```

- univariate polynomials
- non-interactive opening

Formalizing Polynomial Commitments

```

locale abstract_polynomial_commitment_scheme =
  fixes key_gen :: "('ck × 'vk) spmf" — <outputs the keys received by the two parties>
  and commit :: "'ck ⇒ 'r::zero poly ⇒ ('commit × 'trapdoor) spmf"
    — <outputs the commitment as well as the secret, which might be used to derive witnesses,
    and the opening values sent by the committer in the reveal phase>
  and verify_poly :: "'vk ⇒ 'r poly ⇒ 'commit ⇒ 'trapdoor ⇒ bool"
    — <checks whether the polynomial corresponds to the commitment>
  and eval :: "'ck ⇒ 'trapdoor ⇒ 'r poly ⇒ 'argument ⇒ ('evaluation × 'witness)"
    — <outputs a point and a witness>
  and verify_eval :: "'vk ⇒ 'commit ⇒ 'argument ⇒ ('evaluation × 'witness) ⇒ bool"
    — <checks whether the point is on the polynomial corresponding to the commitment>
begin

```

- univariate polynomials
- non-interactive opening

Formalizing Polynomial Commitments

```
definition eval_bind_game :: "('ck, 'commit, 'argument, 'evaluation, 'witness) eval_bind_adversary
  ⇒ bool spmf"
  where "eval_bind_game  $\mathcal{A}$  =
TRY do {
  (ck, vk) ← key_gen;
  (c, i, v, w, v', w') ←  $\mathcal{A}$  ck;
  let b = verify_eval vk c i (v,w);
  let b' = verify_eval vk c i (v',w');
  return_spmf (b ∧ b' ∧ v ≠ v')
} ELSE return_spmf False"
```

```
definition eval_bind_advantage :: "('ck, 'commit, 'argument, 'evaluation, 'witness)
  eval_bind_adversary ⇒ real"
  where "eval_bind_advantage  $\mathcal{A}$  ≡ spmf (eval_bind_game  $\mathcal{A}$ ) True"
```

Formalizing Polynomial Commitments

```

definition eval_bind_game :: "('ck, 'commit, 'argument, 'advantage)
  ⇒ bool spmf"
  where "eval_bind_game  $\mathcal{A}$  =
    TRY do {
      (ck, vk) ← key_gen;
      (c, i, v, w, v', w') ←  $\mathcal{A}$  ck;
      let b = verify_eval vk c i (v,w);
      let b' = verify_eval vk c i (v',w');
      return_spmf (b ∧ b' ∧ v ≠ v')
    } ELSE return_spmf False"

```

```

definition eval_bind_advantage :: "('ck, 'commit, 'argument, 'advantage)
  eval_bind adversary ⇒ real"
  where "eval_bind_advantage  $\mathcal{A}$  ≡ spmf (eval_bind_game  $\mathcal{A}$ ) True"

```

Games:

- polynomial correctness
- polynomial binding
- polynomial hiding
- evaluation correctness
- evaluation binding
- evaluation hiding
- knowledge soundness/
extractability

adversary

two KZG constructions

KZG:

- polynomial correctness
- polynomial binding
- evaluation correctness
- evaluation binding
- (weak) evaluation hiding
- knowledge soundness/
extractability (AGM)

Batch-KZG:

- polynomial correctness
- polynomial binding
- evaluation correctness
- evaluation binding
- knowledge soundness/
extractability (AGM)

two KZG constructions

KZG:

- polynomial correctness
- polynomial binding
- evaluation correctness
- evaluation binding
- (weak) evaluation hiding
- knowledge soundness/
extractability (AGM)

Batch-KZG:

- polynomial correctness
- polynomial binding
- evaluation correctness
- evaluation binding
- knowledge soundness/
extractability (AGM)

two KZG constructions

KZG:

- polynomial correctness
- polynomial binding
- evaluation correctness
- evaluation binding
- (weak) evaluation hiding
- knowledge soundness/
extractability (AGM)

Batch-KZG:

- polynomial correctness
- polynomial binding
- evaluation correctness
- evaluation binding
- knowledge soundness/
extractability (AGM)

KZG Proofs

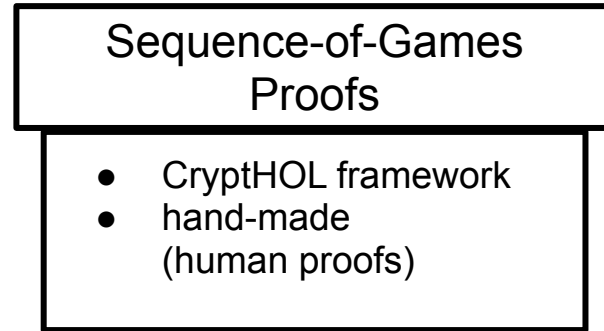
Sequence-of-Games
Proofs

KZG Proofs

Sequence-of-Games
Proofs

$$\text{Attack Game} = \left(\dots \right) \approx \left(\dots \right) \approx \dots \approx \left(\dots \right) = \text{hardness assumption}$$

KZG Proofs



$$\text{Attack Game} = \left(\dots \right) \approx \left(\dots \right) \approx \dots \approx \left(\dots \right) = \text{hardness assumption}$$

KZG Proofs

Sequence-of-Games Proofs

- CryptHOL framework
- hand-made (human proofs)

$$\text{Attack Game} = \left(\dots \right) \approx$$

```

also have "... = TRY do {
  a :: nat ← sample_uniform q;
  x1 :: nat ← sample_uniform q;
  y1 ← sample_uniform q;
  ((m0, m1), σ) ← A1 (x1,y1);
  _ :: unit ← assert_spmf (valid_msg m0 ∧ valid_msg m1);
  d ← coin_spmf;
  let c = ((if d then m0 else m1) + a) mod q;
  b' ← A2 c σ;
  return_spmf (b' = d)} ELSE coin_spmf"
by(simp add: samp_uni_plus_one_time_pad)
also have "... = TRY do {
  x1 :: nat ← sample_uniform q;
  y1 ← sample_uniform q;
  ((m0, m1), σ) ← A1 (x1,y1);
  _ :: unit ← assert_spmf (valid_msg m0 ∧ valid_msg m1);
  d ← coin_spmf;
  c ← map_spmf (λ a. ((if d then m0 else m1) + a) mod q) (sample_uniform q);
  b' ← A2 c σ;
  return_spmf (b' = d)} ELSE coin_spmf"
by(simp add: o_def bind_map_spmf)
also have "... = TRY do {
  x1 :: nat ← sample_uniform q;
  y1 ← sample_uniform q;
  ((m0, m1), σ) ← A1 (x1,y1);
  _ :: unit ← assert_spmf (valid_msg m0 ∧ valid_msg m1);
  d ← coin_spmf;
  c ← sample_uniform q;
  b' :: bool ← A2 c σ;
  return_spmf (b' = d)} ELSE coin_spmf"
by(simp add: samp_uni_plus_one_time_pad)
also have "... = TRY do {
  x1 :: nat ← sample_uniform q;
  y1 ← sample_uniform q;
  ((m0, m1), σ) ← A1 (x1,y1);
  _ :: unit ← assert_spmf (valid_msg m0 ∧ valid_msg m1);
  c :: nat ← sample_uniform q;
  guess :: bool ← A2 c σ;
  map_spmf(=) guess coin_spmf} ELSE coin_spmf"
by(simp add: map_spmf_conv_bind_spmf)

```

the *other* paper

the *other* paper

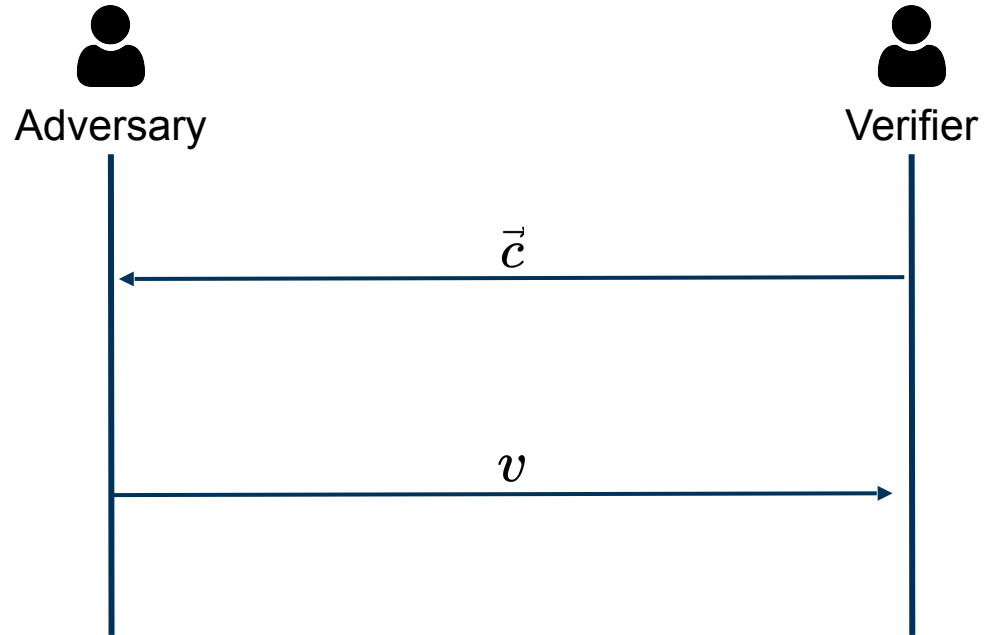
This work:

- batch version (Batch-KZG)
- foundational
- AGM
- knowledge soundness

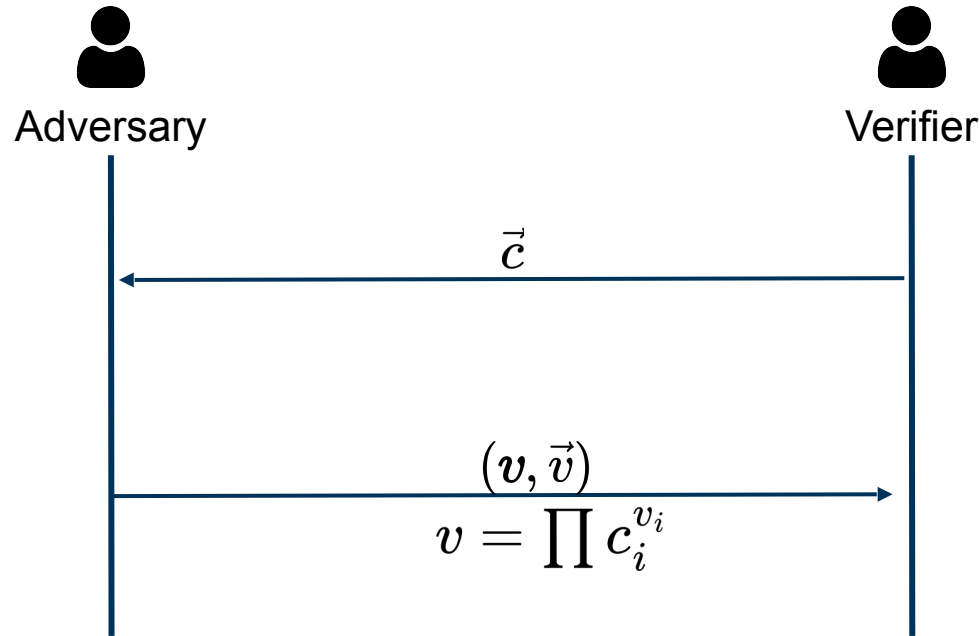
the *other* paper:

- pedersen version (PED-KZG)

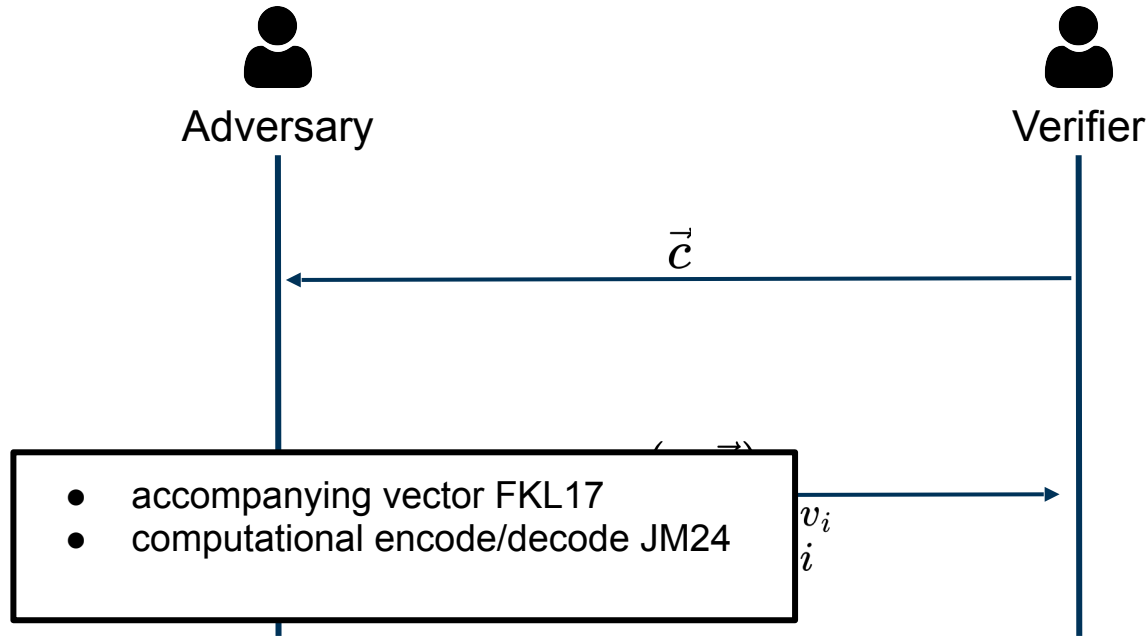
Algebraic Group Model (AGM)



Algebraic Group Model (AGM)

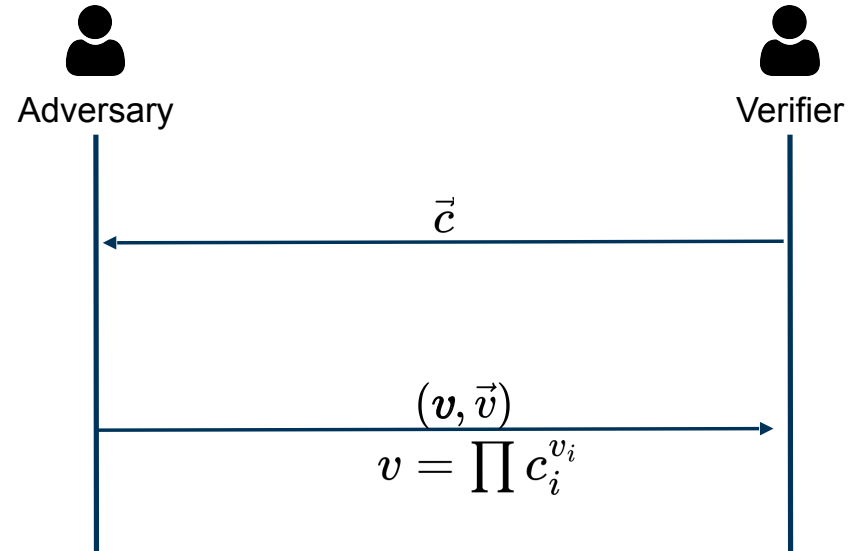


Algebraic Group Model (AGM)



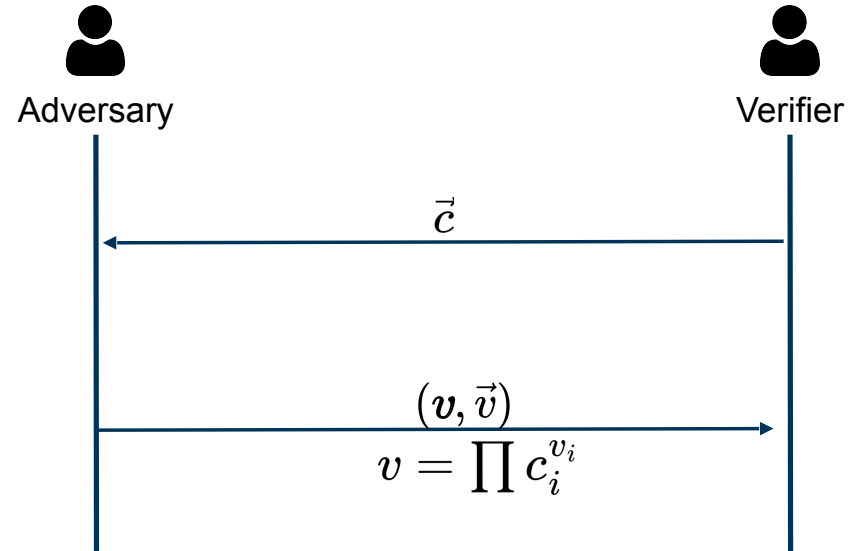
Algebraic Group Model (AGM)

$$\left(\begin{array}{l} \vec{c} \leftarrow \text{Verifier}, \\ v \leftarrow \mathcal{A}(\vec{c}), \\ \vdots \end{array} \right)$$



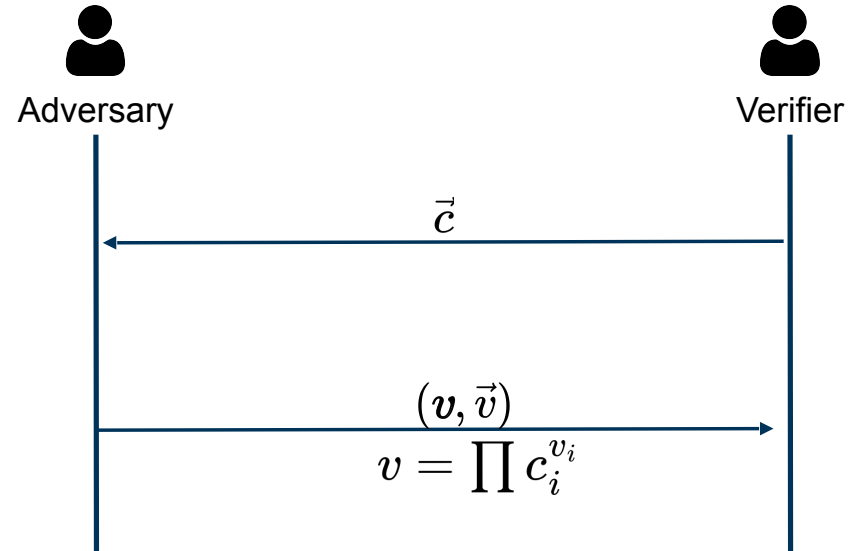
Algebraic Group Model (AGM)

$$\left(\begin{array}{l} \vec{c} \leftarrow \text{Verifier}, \\ (v, \vec{v}) \leftarrow \mathcal{A}(\vec{c}), \\ \dots \wedge v = \prod \vec{v}_i^{\vec{c}_i} \end{array} \right)$$



Weak Algebraic Group Model (AGM)

$$\left(\begin{array}{l} \vec{c} \leftarrow \text{Verifier}, \\ (v, \vec{v}) \leftarrow \mathcal{A}(\vec{c}), \\ \dots \wedge v = \prod \vec{v}_i^{\vec{c}_i} \end{array} \right)$$



Weak Algebraic Group Model (AGM)

$$\left(\begin{array}{l} \vec{c} \leftarrow \text{Verifier}, \\ (v, \vec{v}) \leftarrow \mathcal{A}(\vec{c}), \\ \dots \wedge v = \prod \vec{v}_i^{\vec{c}_i} \end{array} \right)$$

select

constrain

restrict

semi-automatic AGM

$\alpha \Rightarrow \beta \text{ list}$

select

$\beta \text{ list} \Rightarrow \gamma \Rightarrow \text{bool}$

constrain

$(\alpha \text{ spmf} \Rightarrow \gamma) \Rightarrow \alpha \text{ spmf} \Rightarrow \gamma$

restrict

semi-automatic AGM

Adversary Inputs


$$\alpha \Rightarrow \beta \text{ list}$$

$$\beta \text{ list} \Rightarrow \gamma \Rightarrow \text{bool}$$

$$(\alpha \text{ spmf} \Rightarrow \gamma) \Rightarrow \alpha \text{ spmf} \Rightarrow \gamma$$

select

constrain

restrict

semi-automatic AGM

Adversary Inputs

group elements

$$\alpha \Rightarrow \beta \text{ list}$$

$$\beta \text{ list} \Rightarrow \gamma \Rightarrow \text{bool}$$

$$(\alpha \text{ spmf} \Rightarrow \gamma) \Rightarrow \alpha \text{ spmf} \Rightarrow \gamma$$

select

constrain

restrict

semi-automatic AGM

Adversary Inputs

group elements

$$\alpha \Rightarrow \beta \text{ list}$$

$$\beta \text{ list} \Rightarrow \gamma \Rightarrow \text{bool}$$

$$(\alpha \text{ spmf} \Rightarrow \gamma) \Rightarrow \alpha \text{ spmf} \Rightarrow \gamma$$

select

constrain

restrict

semi-automatic AGM

Adversary Inputs

group elements

$\alpha \Rightarrow \beta \text{ list}$

Adversary Output

$\beta \text{ list} \Rightarrow \gamma \Rightarrow \text{bool}$

$(\alpha \text{ spmf} \Rightarrow \gamma) \Rightarrow \alpha \text{ spmf} \Rightarrow \gamma$

select

constrain

restrict

semi-automatic AGM

Adversary Inputs

group elements

$$\alpha \Rightarrow \beta \text{ list}$$

Adversary Output

$$\beta \text{ list} \Rightarrow \gamma \Rightarrow \text{bool}$$

$$(\alpha \text{ spmf} \Rightarrow \gamma) \Rightarrow \alpha \text{ spmf} \Rightarrow \gamma$$

Standard Model
Adversary

select

constrain

restrict

semi-automatic AGM

Adversary Inputs

group elements

$$\alpha \Rightarrow \beta \text{ list}$$

Adversary Output

$$\beta \text{ list} \Rightarrow \gamma \Rightarrow \text{bool}$$

$$(\alpha \text{ spmf} \Rightarrow \gamma) \Rightarrow \alpha \text{ spmf} \Rightarrow \gamma$$

Standard Model
Adversary

AGM Adversary

select

constrain

restrict

semi-automatic AGM

Adversary Inputs

group elements

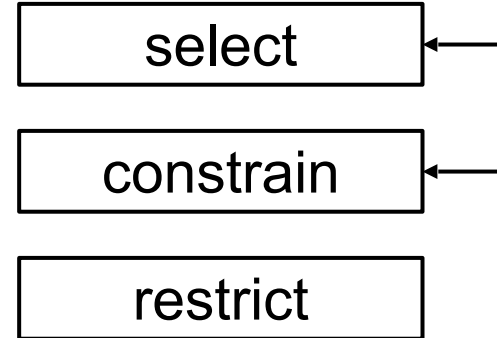
$\alpha \Rightarrow \beta \text{ list}$

Adversary Output

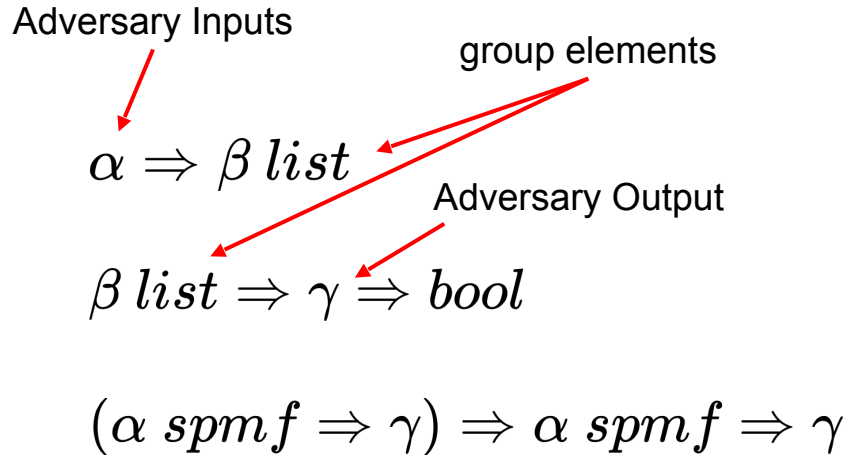
$\beta \text{ list} \Rightarrow \gamma \Rightarrow \text{bool}$

$(\alpha \text{ spmf} \Rightarrow \gamma) \Rightarrow \alpha \text{ spmf} \Rightarrow \gamma$

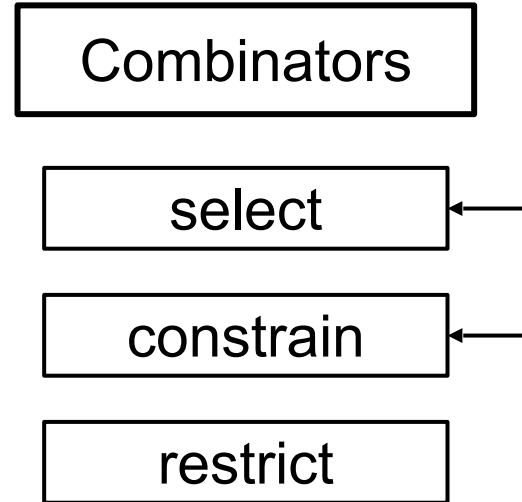
$\Rightarrow$ separation of input and output types!



semi-automatic AGM



$\Rightarrow$ separation of input and output types!



semi-automatic AGM

Combinators

semi-automatic AGM

Primitive instances:

- groupS
- groupC
- noSelect
- noConstrain

Combinators

semi-automatic AGM

Primitive instances:

- groupS
- groupC
- noSelect
- noConstrain

Combinators

Combinator instances:

- listS and listC
- productS and productC
- treeS and treeC
- multisetS and multisetC
- finsetS and finsetC
- finmapS and finmapC

semi-automatic AGM

Primitive instances:

- groupS
- groupC
- noSelect
- noConstrain

Combinators

Combinator instances:

- listS and listC
- productS and productC
- treeS and treeC
- multisetS and multisetC
- finsetS and finsetC
- finmapS and finmapC

Combinators are closed under composition!

semi-automatic AGM

Primitive instances:

- groupS
- groupC
- noSelect
- noConstrain

Combinators

Combinator instances:

- listS and listC
- productS and productC
- treeS and treeC
- multisetS and multisetC
- finsetS and finsetC
- finmapS and finmapC

Combinators are closed under composition!

$\alpha \times (\beta \text{ tree}) \text{ list}$

semi-automatic AGM

Primitive instances:

- groupS
- groupC
- noSelect
- noConstrain

Combinators

Combinator instances:

- listS and listC
- productS and productC
- treeS and treeC
- multisetS and multisetC
- finsetS and finsetC
- finmapS and finmapC

Combinators are closed under composition!

$\alpha \times (\beta \text{ tree}) \text{ list}$

$\implies \text{productS}(\text{noSelect}, \text{listS}(\text{treeS}(\text{groupS})))$

semi-automatic AGM

Primitive instances:

- groupS
- groupC
- noSelect
- noConstrain

⇒ REUSABLE for other algebraic models!!

Combinators

Combinator instances:

- listS and listC
- productS and productC
- treeS and treeC
- multisetS and multisetC
- finsetS and finsetC
- finmapS and finmapC

Combinators are closed under composition!

$\alpha \times (\beta \text{ tree}) \text{ list}$

⇒ `productS(noSelect, listS(treeS(groupS)))`

semi-automatic AGM

Primitive instances:

- groupS
- groupC
- noSelect
- noConstrain

⇒ REUSABLE for other algebraic models!!

E.g. AIM/AGAM, just define your own primitive instance!

Combinators

Combinator instances:

- listS and listC
- productS and productC
- treeS and treeC
- multisetS and multisetC
- finsetS and finsetC
- finmapS and finmapC

Combinators are closed under composition!

$\alpha \times (\beta \text{ tree}) \text{ list}$

⇒ `productS(noSelect, listS(treeS(groupS)))`

Trusted Code Base (TCB)

This work:

- Security and Hardness Games
- Reduction complexity
- Isabelle Kernel

Trusted Code Base (TCB)

This work:

- Security and Hardness Games
- Reduction complexity
- Isabelle Kernel

the *other* paper:

- Security and Hardness Games
- Reduction Complexity
- EasyCrypt Kernel
- EasyCrypt program logic checker
- EasyCrypt SMT translation

What about *Lean*?

This work:

- focus on pairing-based/EC crypto
- “pre-quantum”
- Algebraic Group Model
- polynomial commitments infrastructure
- advanced KZG def and proofs
- $\Rightarrow$ pairing-based SNARKs

***ArkLib/Lean*:**

- focus on ECC/Lattice crypto
- “post-quantum”
- oracle-centric framework
- functional commitments infrastructure
- basic KZG def and security proof
- $\Rightarrow$ ECC/Lattice-based SNARKs

Thank you for your attention!



website

This work:

- Polynomial Commitments in Isabelle
- Security proofs for standard KZG and Batch KZG (incl. knowledge soundness!)
- algebraic computational models in Isabelle